

Aspera Orchestrator Plugin Development Guide

IBM Aspera

2026/06/19

Contents

1	Introduction	2
2	Plugin Structure	3
3	Common Patterns Identified	4
3.1	1. Class Structure	4
3.2	2. Core Methods	4
3.3	3. Execution Patterns	4
4	Required Elements	5
4.1	1. Main Ruby Class	5
4.2	2. Required Methods	5
4.2.1	<code>self.version</code>	5
4.2.2	<code>inputs_spec</code>	5
4.2.3	<code>outputs_spec</code>	5
4.2.4	<code>category</code>	5
4.2.5	<code>execute</code>	6
4.3	3. metadata.yml File	6
5	Data Types	7
6	Execution Status	8
7	Categories	9
8	Simple Example: "Hello World" Plugin	10
8.1	Directory Structure	10
8.2	hello_world_simple.rb	10
8.3	metadata.yml	11
9	How It Works	12
10	Testing Your Plugin	13
11	Advanced Features	14
11.1	Asynchronous Execution	14
11.2	Input Validation	14
11.3	Helper Methods	14
12	Best Practices	15
13	Common Patterns	16
13.1	Using Payload Variables	16
13.2	Remote Node Integration	16
13.3	Multi-Type Conversion	16
14	Conclusion	17

Chapter 1

Introduction

This guide explains how to develop a plugin (action) for Aspera Orchestrator. Plugins allow you to extend Orchestrator's functionality by adding custom actions.

Chapter 2

Plugin Structure

An Aspera Orchestrator plugin consists of several files organized in a dedicated directory:

```
actions/my_plugin/
├── my_plugin.rb          # Main plugin class
├── metadata.yml          # Metadata and version history
├── edit.html.erb         # Configuration UI (optional)
├── help.html.erb         # Help documentation (optional)
└── MyPlugin.png          # Plugin icon (optional)
```

Chapter 3

Common Patterns Identified

After analyzing multiple plugins (DivaArchive, HelloWorld, Filter, LocalExecution), the following common patterns emerge:

3.1 1. Class Structure

- All plugins inherit from `ActiveRecord::Base`
- All plugins include the `Action` module
- Plugins define constants for variable names and default values
- Instance variables `@inputs`, `@outputs`, and `@status` are commonly used

3.2 2. Core Methods

Every plugin implements these essential methods:

- `self.version` - Returns plugin version
- `inputs_spec` - Defines required and optional inputs
- `outputs_spec` - Defines output variables
- `category` - Categorizes the plugin
- `execute` - Contains main execution logic

3.3 3. Execution Patterns

- Synchronous execution: Returns status immediately
- Asynchronous execution: Returns `STATUS_INPROGRESS` and uses `check_status`
- Error handling with try/catch blocks
- Progress reporting via `report_progress`

Chapter 4

Required Elements

4.1 1. Main Ruby Class

Each plugin must:

- Inherit from `ActiveRecord::Base`
- Include the `Action` module
- Define essential methods

4.2 2. Required Methods

4.2.1 `self.version`

Returns the plugin version by reading from `metadata.yml`.

```
def self.version
  return @@plugin_version_loaded if defined?(@@plugin_version_loaded)
  versions = SafeYaml.load_file("#{File.dirname(__FILE__)}/metadata.yml")[:revision_history].map {
    ↪ |change| change[:version].split(".").map { |num| num.to_i } }
  @@plugin_version_loaded = versions.sort_by { |major,minor,revision,patch| [major, minor, revision,
    ↪ (patch.nil? ? 0 : patch)] }.last
  return @@plugin_version_loaded
end
```

4.2.2 `inputs_spec`

Defines required and optional input parameters.

```
def inputs_spec
  required_inputs = { "param1" => TYPE_STRING }
  optional_inputs = { "param2" => TYPE_INT }
  return required_inputs, optional_inputs
end
```

4.2.3 `outputs_spec`

Defines output variables produced by the plugin.

```
def outputs_spec
  return { "result" => TYPE_STRING }
end
```

4.2.4 `category`

Defines the plugin's category in the UI.

```
def category
  return [CATEGORY_FILEOPERATIONS] # or CATEGORY_SYSTEM, CATEGORY_INTEGRATION, etc.
end
```

4.2.5 execute

Contains the main execution logic.

```
def execute
  @outputs = {}
  # Execution logic here
  @outputs["result"] = "value"
  return STATUS_COMPLETE, "Execution successful", @outputs
end
```

4.3 3. metadata.yml File

The `metadata.yml` file contains plugin metadata:

```
---
:revision_history:
  - :author: "Your Name"
    :change_description: Initial release
    :version: 1.0.0
    :date: "2024-01-01"
    :minimum_server_version: 4.1.0

:display_name: My Plugin
:category: File Operations
:help: "Description of what the plugin does"
:plugin_name: MyPlugin
:description: Short description of the plugin
```

Chapter 5

Data Types

Available data types for inputs/outputs:

- `TYPE_STRING` : String
- `TYPE_INT` : Integer
- `TYPE_FLOAT` : Float/Decimal
- `TYPE_FLAG` : Boolean (true/false)
- `TYPE_HASH` : Hash/Dictionary
- `TYPE_ARRAY` : Array

Chapter 6

Execution Status

Possible statuses returned by `execute` :

- `STATUS_COMPLETE` : Successful execution
- `STATUS_FAILED` : Execution failed
- `STATUS_ERROR` : System error
- `STATUS_INPROGRESS` : In progress (for asynchronous execution)
- `STATUS_PAUSED` : Paused

Chapter 7

Categories

Available plugin categories:

- `CATEGORY_FILEOPERATIONS` : File Operations
- `CATEGORY_SYSTEM` : System
- `CATEGORY_INTEGRATION` : Integration
- `CATEGORY_OTHER` : Other Utilities

Chapter 8

Simple Example: "Hello World" Plugin

Here's a minimal plugin example that displays a personalized message:

8.1 Directory Structure

```
actions/hello_world_simple/  
├─ hello_world_simple.rb  
└─ metadata.yml
```

8.2 hello_world_simple.rb

```
require 'yaml'  
  
class HelloWorldSimple < ActiveRecord::Base  
  include Action  
  
  # Output variable name  
  MESSAGE_OUTPUT = "message"  
  
  # Input variable name  
  NAME_INPUT = "name"  
  
  # Version method - reads from metadata.yml  
  def self.version  
    return @@plugin_version_loaded if defined?(@@plugin_version_loaded)  
    versions = SafeYaml.load_file("#{File.dirname(__FILE__)}/metadata.yml")[:revision_history].map {  
      ↪ |change| change[:version].split(".").map { |num| num.to_i } }  
    @@plugin_version_loaded = versions.sort_by { |major, minor, revision, patch| [major, minor,  
      ↪ revision, (patch.nil? ? 0 : patch)] }.last  
    return @@plugin_version_loaded  
  end  
  
  # Define inputs: one required parameter "name"  
  def inputs_spec  
    required_inputs = { NAME_INPUT => TYPE_STRING }  
    optional_inputs = {}  
    return required_inputs, optional_inputs  
  end  
  
  # Define outputs: one output "message"  
  def outputs_spec  
    return { MESSAGE_OUTPUT => TYPE_STRING }  
  end  
  
  # Plugin category  
  def category
```

```

    return [CATEGORY_OTHER]
end

# Main execution method
def execute
  begin
    # Initialize outputs
    @outputs = {}

    # Get the name from inputs
    name = @inputs[NAME_INPUT]

    # Create the message
    message = "Hello, #{name}!"

    # Set the output
    @outputs[MESSAGE_OUTPUT] = message

    # Return success status
    return STATUS_COMPLETE, "Message created successfully", @outputs

  rescue Exception => e
    # Handle errors
    return STATUS_ERROR, "Error: #{e.message}", {}
  end
end
end

```

8.3 metadata.yml

```

---
:revision_history:
  - :author: "Developer"
    :change_description: Initial release
    :version: 1.0.0
    :date: "2024-01-01"
    :minimum_server_version: 4.1.0

:display_name: Hello World Simple
:category: Other Utilities
:help: "This plugin creates a personalized greeting message"
:plugin_name: HelloWorldSimple
:description: A simple plugin that demonstrates the basic structure of an Orchestrator action

```

Chapter 9

How It Works

1. Input: The plugin receives a `name` parameter
2. Processing: It creates a greeting message: "Hello, [name]!"
3. Output: Returns the message in the `message` output variable
4. Status: Returns `STATUS_COMPLETE` on success

Chapter 10

Testing Your Plugin

1. Place your plugin directory in `actions/`
2. Restart Orchestrator
3. The plugin will appear in the action list under the specified category
4. Create a workflow step using your plugin
5. Provide the required inputs
6. Execute and verify the outputs

Chapter 11

Advanced Features

11.1 Asynchronous Execution

For long-running operations, implement asynchronous execution:

```
def synchronous_execution?  
  return false # Indicates asynchronous execution  
end  
  
def execute  
  # Start the operation  
  @status = STATUS_INPROGRESS  
  # ... start async operation ...  
  return @status, "Operation started", @outputs  
end  
  
def check_status  
  # Check operation status  
  # ... check if operation is complete ...  
  if operation_complete?  
    return STATUS_COMPLETE, "Operation finished", @outputs  
  else  
    return STATUS_INPROGRESS, "Still processing", polling_interval  
  end  
end
```

11.2 Input Validation

```
def validate_inputs(inputs_hash = @inputs)  
  # Custom validation logic  
  return false if inputs_hash["required_param"].nil?  
  return false if inputs_hash["number"].to_i < 0  
  return true  
end
```

11.3 Helper Methods

Use helper methods to access configuration values:

```
def default_get(field_name)  
  # Gets value from inputs or falls back to configuration  
  return @inputs[VAR_NAME] if @inputs && @inputs[VAR_NAME].present?  
  return self.send(field_name) if self.respond_to?(field_name)  
  return nil  
end
```

Chapter 12

Best Practices

1. Error Handling: Always wrap execution logic in try/catch blocks
2. Logging: Use `Rails.logger` for debugging and error messages
3. Constants: Define constants for variable names and default values
4. Documentation: Provide clear help text and parameter descriptions
5. Versioning: Update `metadata.yml` with each change
6. Testing: Test with various input combinations
7. Cleanup: Clean up temporary files and resources
8. Progress Reporting: Use `report_progress` for long operations

Chapter 13

Common Patterns

13.1 Using Payload Variables

```
def inputs_spec
  required_inputs = {}
  # Extract variables from a payload string
  Payload.variables(command).each do |var_name|
    required_inputs[var_name] = TYPE_STRING
  end
  return required_inputs, {}
end

def execute
  # Expand payload with actual values
  expanded_command = Payload.expand(command, @inputs, self)
  # ... use expanded_command ...
end
```

13.2 Remote Node Integration

```
def dependencies
  return [{
    entity: "RemoteNode",
    id_key: "name",
    id_value: remote_node,
    dependent_field: "remote_node"
  }]
end
```

13.3 Multi-Type Conversion

```
# Convert string to typed value
value = MultiType.convert_value(string_value, TYPE_INT)

# Convert value to string
string = MultiType.convert_to_string(value)

# Convert hash representation
hash = MultiType.convert_value_hash(string_hash)
```

Chapter 14

Conclusion

This guide provides the foundation for developing Aspera Orchestrator plugins. Start with the simple example and gradually add complexity as needed. Refer to existing plugins in [actions/](#) for more advanced patterns and techniques.

End of document